

C++ Programming

Day 4: 참조와 동적 메모리

Reference, Stack and Heap, Dynamic Allocation, Dynamic Array

Contents

1	학습 목표	3
2	참조란 무엇인가	3
2.1	참조 선언 문법	3
3	값 복사와 참조의 차이	4
3.1	메모리 관점에서 비교	5
4	참조의 초기화 규칙	5
4.1	선언과 동시에 초기화해야 한다	5
4.2	참조는 다른 변수로 재연결할 수 없다	5
4.3	참조와 포인터의 차이	6
5	함수의 값 전달	6
5.1	값 전달의 메모리 구조	7
6	참조 전달	7
6.1	값 전달과 참조 전달 비교	7
7	참조를 이용한 swap	8
8	스택 메모리와 힙 메모리	8
8.1	스택 메모리	9
8.2	힙 메모리	9
9	동적 메모리 할당: new	9
9.1	동적으로 할당한 값 사용	9
9.2	할당과 동시에 초기화	10
10	동적 메모리 해제: delete	10
10.1	해제 후 포인터 상태	10
11	nullptr	10
12	댕글링 포인터	11
12.1	댕글링 포인터 사용의 위험	11
12.2	예방 방법	11
13	메모리 누수	12
13.1	주소를 덮어쓰는 경우	12
13.2	반복문 안의 메모리 누수	12
14	동적 배열	12
14.1	동적 배열 원소 접근	13
14.2	delete[]로 해제	13
15	동적 배열 초기화와 입력	13
16	배열과 포인터의 관계	14
16.1	인덱스와 역참조	14
16.2	주소가 증가하는 단위	15

17 포인터 연산으로 배열 순회	15
17.1 배열 범위를 벗어나면 안 된다	16
18 동적 메모리를 함수에서 사용하기	16
19 동적 메모리 사용 원칙	17
20 실습 목록	18
21 핵심 요약	18

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 참조(reference)의 개념과 별칭(alias)의 의미
- 값 복사와 참조 연결의 차이
- 참조의 초기화 규칙과 재연결 불가능성
- 값 전달과 참조 전달의 차이
- 참조를 이용한 swap 함수 구현
- 스택 메모리와 힙 메모리의 차이
- new, delete, nullptr의 사용
- dangling 포인터와 메모리 누수의 원인
- 동적 배열과 new[], delete[]
- 배열과 포인터 연산의 관계

2. 참조란 무엇인가

참조(reference)는 이미 존재하는 변수에 붙이는 또 하나의 이름이다. 새로운 값을 저장하는 독립적인 변수를 만드는 것이 아니라, 원래 변수와 같은 메모리 공간을 가리키는 별칭(alias)을 만든다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int number = 10;
8     int& ref = number;
9
10    cout << number << endl;
11    cout << ref << endl;
12
13    return 0;
14 }
```

출력 결과:

```

10
10
```

ref는 number의 복사본이 아니다. 두 이름은 같은 정수 객체를 나타낸다.

2.1. 참조 선언 문법

참조는 자료형 뒤에 &를 붙여 선언한다.

```

1 int value = 20;
2 int& reference = value;
```

여기서 `int&`는 “정수형 변수에 대한 참조”라는 뜻이다.

핵심 포인트

선언문에서의 `&`는 참조를 나타낸다. 표현식에서의 `&variable`은 해당 변수의 주소를 구하는 주소 연산자이다. 모양은 같지만 사용되는 위치와 의미가 다르다.

3. 값 복사와 참조의 차이

일반 변수를 다른 변수로 초기화하면 값이 복사된다.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int original = 10;
8     int copied = original;
9
10    copied = 50;
11
12    cout << original << endl;
13    cout << copied << endl;
14
15    return 0;
16 }
```

출력 결과:

```
10
50
```

`original`과 `copied`는 서로 다른 메모리 공간을 가진다. 따라서 `copied`를 수정해도 `original`은 변하지 않는다.

반면 참조는 원본과 같은 메모리 공간을 사용한다.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int original = 10;
8     int& ref = original;
9
10    ref = 50;
11
12    cout << original << endl;
13    cout << ref << endl;
14
15    return 0;
16 }
```

출력 결과:

```
50
50
```

참조를 통해 값을 변경하면 원본도 함께 변경된다. 사실 두 값이 동시에 바뀐 것이 아니라 하나의 객체를 두 이름으로 접근한 것이다.

3.1. 메모리 관점에서 비교

구분	값 복사	참조
선언	<code>int b = a;</code>	<code>int& b = a;</code>
메모리 공간	서로 다름	동일함
b 수정 시 a	변하지 않음	함께 변함
독립된 객체 여부	독립된 객체	원본의 별칭

주소를 출력하면 차이를 더 분명하게 확인할 수 있다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int original = 10;
8     int copied = original;
9     int& ref = original;
10
11     cout << &original << endl;
12     cout << &copied << endl;
13     cout << &ref << endl;
14
15     return 0;
16 }
```

original과 ref의 주소는 같고, copied의 주소는 다르다.

4. 참조의 초기화 규칙

4.1. 선언과 동시에 초기화해야 한다

참조는 반드시 선언하는 순간 연결할 변수를 지정해야 한다.

```

1 int number = 10;
2 int& ref = number;
```

다음과 같이 참조만 선언하는 것은 허용되지 않는다.

```

1 int& ref;
```

참조는 독립적인 저장 공간이 아니라 기존 객체의 별칭이므로, 처음부터 어떤 객체의 별칭인지 정해져 있어야 한다.

4.2. 참조는 다른 변수로 재연결할 수 없다

한 번 연결된 참조는 다른 변수의 참조로 바뀌지 않는다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int first = 10;
```

```

8  int second = 20;
9  int& ref = first;
10
11  ref = second;
12
13  cout << first << endl;
14  cout << second << endl;
15  cout << ref << endl;
16
17  return 0;
18 }

```

출력 결과:

```

20
20
20

```

`ref = second;`는 참조를 `second`로 다시 연결하는 문장이 아니다. `second`의 값을 `ref`가 가리키는 원본, 즉 `first`에 대입한다.

4.3. 참조와 포인터의 차이

참조와 포인터는 모두 원본 객체에 간접적으로 접근할 수 있지만 사용 방식이 다르다.

구분	참조	포인터
선언	<code>int& ref = value;</code>	<code>int* ptr = &value;</code>
초기화	반드시 필요	나중에 가능
다른 객체로 변경	불가능	가능
널 상태	일반적으로 없음	<code>nullptr</code> 가능
값 접근	<code>ref</code>	<code>*ptr</code>

참조는 일반 변수처럼 간단하게 사용할 수 있고, 포인터는 주소 자체를 저장하고 변경할 수 있다는 차이가 있다.

5. 함수의 값 전달

함수의 매개변수를 일반 변수로 선언하면 인자의 값이 복사된다. 이를 값 전달(`pass by value`)이라고 한다.

```

1  #include <iostream>
2
3  using namespace std;
4
5  void increase(int number)
6  {
7      number++;
8  }
9
10 int main()
11 {
12     int value = 10;
13
14     increase(value);
15
16     cout << value << endl;
17 }

```

```

18     return 0;
19 }

```

출력 결과:

10

increase()의 number는 value의 복사본이다. 함수 내부에서 복사본을 수정했으므로 원본은 변하지 않는다.

5.1. 값 전달의 메모리 구조

함수를 호출하면 매개변수는 함수 내부에서 사용하는 새로운 지역 변수로 만들어진다.

```

main의 value:      10
increase의 number: 10

```

두 변수의 초기값은 같지만 서로 다른 객체이다.

6. 참조 전달

매개변수를 참조로 선언하면 인자의 복사본을 만들지 않고 원본에 직접 접근한다. 이를 참조 전달(pass by reference)이라고 한다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 void increase(int& number)
6 {
7     number++;
8 }
9
10 int main()
11 {
12     int value = 10;
13
14     increase(value);
15
16     cout << value << endl;
17
18     return 0;
19 }

```

출력 결과:

11

함수의 number는 main()의 value에 대한 참조이다. 따라서 함수 안의 변경이 원본에 반영된다.

6.1. 값 전달과 참조 전달 비교

구분	값 전달	참조 전달
매개변수	int number	int& number
인자 복사	발생	발생하지 않음
원본 수정	불가능	가능
큰 객체 전달 비용	상대적으로 큼	상대적으로 작음

핵심 포인트

참조 전달은 원본을 수정해야 할 때 유용하다. 다만 함수 호출만 보고 원본이 바뀌는지 알기 어려울 수 있으므로, 함수 이름과 역할을 명확하게 정하는 것이 중요하다.

7. 참조를 이용한 swap

두 변수의 값을 바꾸는 함수를 값 전달로 작성하면 원하는 결과를 얻을 수 없다.

```
1 void wrongSwap(int a, int b)
2 {
3     int temp = a;
4     a = b;
5     b = temp;
6 }
```

위 함수에서 바뀌는 것은 매개변수 a, b뿐이다. 호출한 쪽의 원본 변수는 변하지 않는다. 참조 전달을 사용하면 원본 변수의 값을 직접 바꿀 수 있다.

```
1 #include <iostream>
2
3 using namespace std;
4
5 void swapValues(int& a, int& b)
6 {
7     int temp = a;
8     a = b;
9     b = temp;
10 }
11
12 int main()
13 {
14     int x = 10;
15     int y = 20;
16
17     swapValues(x, y);
18
19     cout << x << endl;
20     cout << y << endl;
21
22     return 0;
23 }
```

출력 결과:

```
20
10
```

호출 시점에는 일반 변수처럼 `swapValues(x, y)`라고 작성하지만, 함수 정의에서 매개변수가 참조로 선언되어 있으므로 a와 b는 각각 x와 y의 별칭이 된다.

8. 스택 메모리와 힙 메모리

프로그램이 실행되면 필요한 데이터가 메모리에 저장된다. C++에서 지역 변수와 동적 할당을 이해하려면 스택(stack)과 힙(heap)의 차이를 알아야 한다.

8.1. 스택 메모리

함수 내부에서 선언한 지역 변수는 일반적으로 스택 메모리에 저장된다.

```
1 void function()
2 {
3     int number = 10;
4     double value = 3.14;
5 }
```

함수가 호출되면 지역 변수가 만들어지고, 함수가 끝나면 자동으로 제거된다. 프로그래머가 직접 메모리를 해제할 필요가 없다.

8.2. 힙 메모리

힙은 프로그램 실행 중 필요한 크기만큼 직접 메모리를 요청할 때 사용하는 영역이다. C++에서는 `new` 연산자로 힙 메모리를 할당하고, `delete` 연산자로 해제한다.

구분	스택	힙
생성 방식	지역 변수 선언	<code>new</code> 사용
해제 방식	자동	<code>delete</code> 필요
수명	블록 또는 함수가 끝날 때까지	직접 해제할 때까지
관리 주체	컴파일러와 실행 환경	프로그래머
주요 위험	제한된 크기	누수, 댕글링 포인터

핵심 포인트

스택과 힙의 핵심 차이는 “자동 관리”와 “직접 관리”이다. 힙 메모리는 필요한 시점과 크기를 유연하게 정할 수 있지만, 반드시 올바르게 해제해야 한다.

9. 동적 메모리 할당: `new`

`new`는 힙에 메모리를 할당하고, 할당된 메모리의 주소를 반환한다. 따라서 반환된 주소를 저장할 포인터가 필요하다.

```
1 int* ptr = new int;
```

위 코드는 힙에 정수 하나를 저장할 공간을 만들고, 그 주소를 `ptr`에 저장한다.

9.1. 동적으로 할당한 값 사용

포인터가 가리키는 값에 접근하려면 역참조 연산자 `*`를 사용한다.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int* ptr = new int;
8
9     *ptr = 42;
10
11     cout << *ptr << endl;
12
13     delete ptr;
14 }
```

```

15     return 0;
16 }

```

ptr 자체는 주소를 저장하는 포인터 변수이고, *ptr은 힙에 실제로 저장된 정수이다.

9.2. 할당과 동시에 초기화

```

1 int* ptr = new int(42);

```

중괄호 초기화도 사용할 수 있다.

```

1 int* ptr = new int{42};

```

두 경우 모두 정수 공간을 만들고 초기값 42를 저장한다.

10. 동적 메모리 해제: delete

new로 할당한 메모리는 사용이 끝난 뒤 delete로 해제해야 한다.

```

1 int* ptr = new int(42);
2
3 cout << *ptr << endl;
4
5 delete ptr;

```

delete ptr;는 ptr 변수를 제거하는 것이 아니다. ptr이 가리키는 힙 메모리를 해제한다. 포인터 변수 자체는 해당 블록이 끝날 때까지 남아 있다.

10.1. 해제 후 포인터 상태

메모리를 해제한 뒤에도 포인터에는 이전 주소가 남아 있다. 그러나 그 주소의 메모리는 더 이상 사용할 수 없다.

```

1 int* ptr = new int(42);
2
3 delete ptr;
4
5 cout << *ptr << endl;

```

위 코드의 마지막 줄은 정의되지 않은 동작(undefined behavior)을 일으킨다. 프로그램이 우연히 값을 출력할 수도 있고, 오류가 발생할 수도 있다.

11. nullptr

nullptr는 포인터가 유효한 객체를 가리키고 있지 않음을 나타내는 값이다.

```

1 int* ptr = nullptr;

```

포인터를 아직 어떤 주소와도 연결하지 않을 때 초기값으로 사용할 수 있다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int* ptr = nullptr;
8
9     if (ptr == nullptr)

```

```

10 {
11     cout << "No object" << endl;
12 }
13
14 return 0;
15 }

```

메모리를 해제한 뒤 포인터를 nullptr로 바꾸면 잘못된 주소를 다시 사용하는 실수를 줄일 수 있다.

```

1 int* ptr = new int(42);
2
3 delete ptr;
4 ptr = nullptr;

```

핵심 포인트

delete nullptr;는 안전하다. 포인터가 nullptr인지 검사한 뒤에만 삭제해야 하는 것은 아니지만, 포인터가 현재 유효한 메모리를 가리키는지는 항상 구분해야 한다.

12. 댕글링 포인터

댕글링 포인터(dangling pointer)는 이미 수명이 끝났거나 해제된 메모리의 주소를 계속 저장하고 있는 포인터이다.

```

1 int* ptr = new int(42);
2
3 delete ptr;

```

delete 이후 ptr은 더 이상 유효한 정수를 가리키지 않지만 이전 주소는 그대로 가지고 있다. 이 상태의 ptr이 댕글링 포인터이다.

12.1. 댕글링 포인터 사용의 위험

```

1 int* ptr = new int(42);
2
3 delete ptr;
4
5 *ptr = 100;

```

이미 해제된 메모리에 값을 쓰려고 하므로 정의되지 않은 동작이 발생한다. 다른 데이터가 손상되거나 프로그램이 비정상 종료될 수 있다.

12.2. 예방 방법

가장 기본적인 방법은 해제 직후 포인터를 nullptr로 만드는 것이다.

```

1 delete ptr;
2 ptr = nullptr;

```

또한 여러 포인터가 같은 동적 메모리를 가리킬 때는 한 포인터가 메모리를 해제한 뒤 다른 포인터도 모두 무효가 된다는 점을 주의해야 한다.

```

1 int* first = new int(42);
2 int* second = first;
3
4 delete first;
5 first = nullptr;

```

이때 second는 여전히 이전 주소를 저장하고 있으므로 댕글링 포인터이다.

13. 메모리 누수

메모리 누수(memory leak)는 동적으로 할당한 메모리를 더 이상 사용할 방법이 없는데도 해제하지 않은 상태를 말한다.

```
1 void createNumber()
2 {
3     int* ptr = new int(42);
4 }
```

함수가 끝나면 지역 포인터 `ptr`은 사라진다. 하지만 `new`로 만든 힙 메모리는 자동으로 해제되지 않는다. 그 주소를 잃어버렸으므로 이후에는 `delete`할 수도 없다.

13.1. 주소를 덮어쓰는 경우

```
1 int* ptr = new int(10);
2
3 ptr = new int(20);
```

두 번째 대입이 실행되면 첫 번째로 할당한 정수의 주소를 잃어버린다. 첫 번째 메모리는 프로그램이 종료될 때까지 남아 메모리 누수가 된다.

올바른 순서는 기존 메모리를 먼저 해제한 뒤 새로운 메모리를 할당하는 것이다.

```
1 int* ptr = new int(10);
2
3 delete ptr;
4 ptr = new int(20);
5
6 delete ptr;
7 ptr = nullptr;
```

13.2. 반복문 안의 메모리 누수

작은 누수도 반복되면 큰 문제가 된다.

```
1 for (int i = 0; i < 1000000; i++)
2 {
3     int* ptr = new int(i);
4 }
```

각 반복에서 포인터 변수는 사라지지만 할당된 정수는 해제되지 않는다. 반복문이 진행될수록 사용 중인 메모리가 계속 증가한다.

14. 동적 배열

일반 배열의 크기는 선언 시점에 정해져야 한다. 실행 중 입력받은 크기로 배열을 만들고 싶을 때 동적 배열을 사용할 수 있다.

```
1 int size;
2 cin >> size;
3
4 int* numbers = new int[size];
```

`new int[size]`는 정수 `size`개가 연속해서 저장될 공간을 힙에 만들고 첫 번째 원소의 주소를 반환한다.

14.1. 동적 배열 원소 접근

동적 배열도 일반 배열과 같은 대괄호 문법으로 접근할 수 있다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int size;
8
9     cout << "Size: ";
10    cin >> size;
11
12    int* numbers = new int[size];
13
14    for (int i = 0; i < size; i++)
15    {
16        numbers[i] = (i + 1) * 10;
17    }
18
19    for (int i = 0; i < size; i++)
20    {
21        cout << numbers[i] << ' ';
22    }
23
24    cout << endl;
25
26    delete[] numbers;
27    numbers = nullptr;
28
29    return 0;
30 }
```

14.2. delete[]로 해제

new[]로 만든 배열은 반드시 delete[]로 해제해야 한다.

```

1 int* numbers = new int[5];
2
3 delete[] numbers;
4 numbers = nullptr;
```

단일 객체와 배열의 할당 및 해제 방식은 짝을 맞춰야 한다.

대상	할당	해제
단일 객체	new int	delete ptr
배열	new int[n]	delete[] ptr

핵심 포인트

new[]에 delete를 사용하거나, 단일 객체에 delete[]를 사용하는 것은 잘못된 코드이다. 할당 방식과 해제 방식은 반드시 정확히 대응해야 한다.

15. 동적 배열 초기화와 입력

동적 배열도 생성과 동시에 값 초기화를 할 수 있다.

```
1 int* numbers = new int[5]{};
```

빈 중괄호를 사용하면 모든 정수 원소가 0으로 초기화된다.
일부 값을 직접 지정할 수도 있다.

```
1 int* numbers = new int[5]{10, 20, 30, 40, 50};
```

사용자로부터 원소를 입력받는 예제는 다음과 같다.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int size;
8     int sum = 0;
9
10    cout << "Size: ";
11    cin >> size;
12
13    int* numbers = new int[size];
14
15    for (int i = 0; i < size; i++)
16    {
17        cout << "numbers[" << i << "]: ";
18        cin >> numbers[i];
19        sum += numbers[i];
20    }
21
22    cout << "Sum: " << sum << endl;
23
24    delete[] numbers;
25    numbers = nullptr;
26
27    return 0;
28 }
```

동적 배열은 사용이 끝난 지점에서 즉시 해제하는 습관이 중요하다.

16. 배열과 포인터의 관계

배열 이름은 대부분의 표현식에서 첫 번째 원소의 주소처럼 사용된다.

```
1 int numbers[5] = {10, 20, 30, 40, 50};
2
3 cout << numbers << endl;
4 cout << &numbers[0] << endl;
```

두 출력은 같은 주소를 나타낸다.

16.1. 인덱스와 역참조

배열의 인덱스 표현은 포인터 연산으로 바꾸어 쓸 수 있다.

```
1 numbers[i]
2 *(numbers + i)
```

두 표현은 같은 원소를 나타낸다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int numbers[5] = {10, 20, 30, 40, 50};
8
9     cout << numbers[2] << endl;
10    cout << *(numbers + 2) << endl;
11
12    return 0;
13 }

```

출력 결과:

```

30
30

```

16.2. 주소가 증가하는 단위

정수 포인터에 1을 더하면 주소가 1바이트 증가하는 것이 아니라 다음 정수의 주소로 이동한다.

```

1 int numbers[3] = {10, 20, 30};
2 int* ptr = numbers;
3
4 cout << *ptr << endl;
5 ptr++;
6 cout << *ptr << endl;

```

첫 번째 출력은 10이고, ptr++ 이후 두 번째 출력은 20이다. 포인터 연산은 가리키는 자료형의 크기를 기준으로 이동한다.

17. 포인터 연산으로 배열 순회

배열을 인덱스로 순회하는 대신 포인터를 이동시키며 순회할 수 있다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int numbers[5] = {10, 20, 30, 40, 50};
8     int* ptr = numbers;
9
10    for (int i = 0; i < 5; i++)
11    {
12        cout << *ptr << ' ';
13        ptr++;
14    }
15
16    cout << endl;
17
18    return 0;
19 }

```

동적 배열에도 같은 원리가 적용된다.


```

1 int* numbers = new int[5]{10, 20, 30, 40, 50};
2
3 for (int i = 0; i < 5; i++)
4 {
5     cout << *(numbers + i) << ' ';
6 }
7
8 delete[] numbers;
9 numbers = nullptr;

```

17.1. 배열 범위를 벗어나면 안 된다

포인터 연산으로 배열 범위를 넘어선 위치를 역참조하면 정의되지 않은 동작이 발생한다.

```

1 int numbers[3] = {10, 20, 30};
2
3 cout << *(numbers + 3) << endl;

```

유효한 인덱스는 0, 1, 2뿐이다. `numbers + 3`은 배열의 마지막 원소 다음 위치를 가리키며, 그 위치를 역참조해서는 안 된다.

18. 동적 메모리를 함수에서 사용하기

함수 안에서 동적 배열을 만들고 주소를 반환할 수 있다.

```

1 int* createArray(int size)
2 {
3     int* numbers = new int[size]{};
4     return numbers;
5 }

```

함수 내부의 지역 포인터 `numbers`는 함수가 끝나면 사라지지만, 힙에 생성한 배열은 자동으로 사라지지 않는다. 주소가 호출한 쪽으로 반환되므로 계속 사용할 수 있다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int* createArray(int size)
6 {
7     int* numbers = new int[size]{};
8     return numbers;
9 }
10
11 int main()
12 {
13     int size = 5;
14     int* values = createArray(size);
15
16     for (int i = 0; i < size; i++)
17     {
18         values[i] = i + 1;
19     }
20
21     for (int i = 0; i < size; i++)
22     {
23         cout << values[i] << ' ';
24     }

```

```

25     cout << endl;
26
27     delete[] values;
28     values = nullptr;
29
30     return 0;
31 }
32

```

메모리를 생성한 함수와 해제하는 함수가 다를 수 있으므로, 어떤 코드가 해제 책임을 가지는지 분명하게 정해야 한다.

19. 동적 메모리 사용 원칙

동적 메모리를 안전하게 사용하려면 다음 순서를 습관화해야 한다.

1. 포인터를 선언하고 필요하면 `nullptr`로 초기화한다.
2. `new` 또는 `new[]`로 메모리를 할당한다.
3. 포인터를 통해 데이터를 사용한다.
4. 더 이상 필요하지 않을 때 `delete` 또는 `delete[]`로 해제한다.
5. 해제한 포인터를 `nullptr`로 바꾼다.

```

1 int* ptr = nullptr;
2
3 ptr = new int(42);
4
5 cout << *ptr << endl;
6
7 delete ptr;
8 ptr = nullptr;

```

동적 배열도 같은 흐름을 따른다.

```

1 int* numbers = nullptr;
2
3 numbers = new int[5]{};
4
5 for (int i = 0; i < 5; i++)
6 {
7     numbers[i] = i * 10;
8 }
9
10 delete[] numbers;
11 numbers = nullptr;

```

20. 실습 목록

실습

Day 4 실습 파일은 다음과 같이 관리한다.

- `practice/p01_reference.cpp`
- `practice/p02_dynamic.cpp`
- `practice/p03_combined.cpp`

21. 핵심 요약

1. 참조는 기존 변수에 붙이는 별칭이며 원본과 같은 객체를 나타낸다.
2. 참조는 선언과 동시에 초기화해야 하고, 이후 다른 변수로 재연결할 수 없다.
3. 값 전달은 인자를 복사하고, 참조 전달은 함수가 원본에 직접 접근하게 한다.
4. 두 원본 변수의 값을 바꾸려면 `swap` 함수의 매개변수를 참조로 선언해야 한다.
5. 지역 변수는 일반적으로 스택에 저장되고 수명이 끝나면 자동으로 제거된다.
6. `new`로 만든 힙 메모리는 `delete`로 직접 해제해야 한다.
7. 해제된 메모리를 가리키는 포인터는 덩글링 포인터가 되며 사용해서는 안 된다.
8. 동적 메모리의 주소를 잃어버리고 해제하지 않으면 메모리 누수가 발생한다.
9. `new[]`로 만든 배열은 반드시 `delete[]`로 해제해야 한다.
10. 배열의 `arr[i]`는 포인터 표현 `*(arr + i)`와 같은 의미이다.